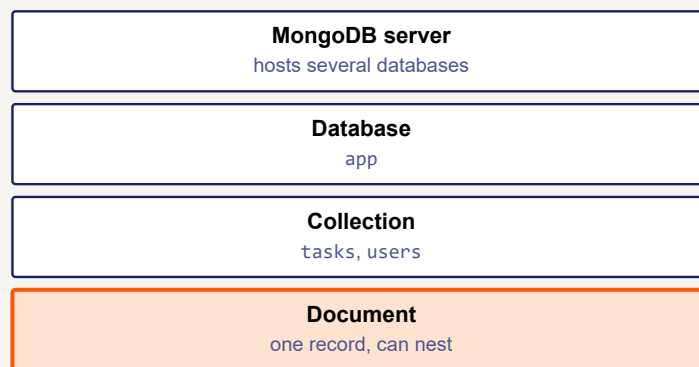


1. **Documents and collections:** Databases, collections and documents · BSON, `_id` and ObjectId · Flexible schema, and who checks the data
2. **CRUD operations:** Creating documents: `insertOne` and `insertMany` · Reading: `find`, `findOne` and the filter · Updating: `updateOne`, `updateMany` and update operators · Deleting, and the danger of `{}`
3. **Queries:** Query operators: comparison and logic · Dot notation: inside sub-documents and arrays · Projection: choose the fields you get back · Sorting and pagination: `sort`, `skip`, `limit`
4. **Embedding and referencing:** Embedding: keep together what you read together · Referencing: store the `_id` of another document · Embed or reference? Rules of thumb
5. **Aggregation:** The aggregation pipeline · `$group`: one result per group · `$unwind` and `$lookup`
6. **Indexes and the back end:** Indexes: from scanning to looking up · Connecting from Node.js: the official driver · Mongoose: schemas and models on top of the driver · SQL or MongoDB? Choosing honestly · From SQL to MongoDB: the vocabulary

Documents and collections

Databases, collections and documents

MongoDB is a document database: each record is a **document** (a JSON-like object that can contain sub-objects and arrays), similar documents live together in a **collection**, and collections are grouped in a **database**.



A server holds databases, a database holds collections, a collection holds documents.

Document One record, like a row, except that it can nest objects and arrays.

Field A named value inside a document, like a column, but not declared in advance and of any type.

Collection A group of documents, usually of the same kind, like a table.

Database A group of collections, like a schema in PostgreSQL. One MongoDB server can host several.

mongosh

```
// one document of the tasks collection (as the shell prints it)
{
  _id: ObjectId('6a9a8890a1b2c3d4e5000104'),
  title: 'Set up CI',
  done: false,
  priority: 5,
  // an array inside the document
  tags: [ 'devops', 'urgent' ],
  // a sub-document
  owner: { name: 'Ben', email: 'ben@example.com' },
  createdAt: ISODate('2026-09-04T09:00:00.000Z'),
  due: ISODate('2026-09-20T09:00:00.000Z')
}
```

Example. The playground's database app has two collections: `tasks` (10 documents) and `users` (3). The task in this card's code keeps its two tags and its owner inside itself. In the [relational schema](#) the same information is spread over three tables (`tasks`, `users` and a `tags` table), joined by keys. Run `show collections` in the playground, then `db.tasks.find()`.

Common mistake. Misspelling a collection name and trusting the empty result. `db.task.find()` (no "s") returns nothing and raises no error, and `db.task.insertOne(...)` silently creates a second collection called `task`. When a query returns nothing unexpectedly, check the name first with `show collections`.

BSON, `_id` and ObjectId

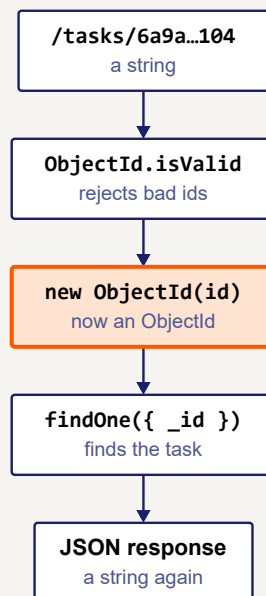
MongoDB stores documents as **BSON** (binary JSON), which adds types JSON lacks, such as dates and **ObjectId**; every document has an `_id` field, unique in its collection, which by default is an **ObjectId**.

The BSON types you will meet

BSON type	Shell literal	In Node.js	In JSON (API response)
String	'Set up CI'	'Set up CI'	"Set up CI"
Double / Int32	5, 2.5	5, 2.5	5, 2.5
Date	ISODate('2026-09-04')	new Date('2026-09-04')	"2026-09-04T00:00:00.000Z" (a string)
ObjectId	ObjectId('6a9a...104')	new ObjectId('6a9a...104')	"6a9a...104" (a string)

The `_id` field

- **Every document has one:** `_id` is the primary key (see [Primary keys](#)). It is unique in the collection and cannot change after the insert.
- **An ObjectId by default:** if you give none, the driver generates a 12-byte ObjectId, written as 24 hexadecimal characters.
- **Made anywhere:** its first 4 bytes are the creation time in seconds, then 5 random bytes and a 3-byte counter, so any machine can create one without asking the database, and ids sort roughly by creation time.



An id leaves the API as a string and must come back as an ObjectId.

```

// JSON has no date type: a Date survives a round trip only as a string.
const task = { title: 'Set up CI', createdAt: new Date('2026-09-04T09:00:00Z') };
const text = JSON.stringify(task);
console.log(text);
const back = JSON.parse(text);
console.log(typeof back.createdAt);
// → 'string', not a Date any more

```

```
console.log(back.createdAt instanceof Date); // false
// That is why BSON (and Extended JSON) add real date and ObjectId types.
```

Example. Ben's user document has `_id: ObjectId('6a93ea08a1b2c3d4e5000002')`. In the playground, `db.users.findOne({ _id: ObjectId('6a93ea08a1b2c3d4e5000002') })` returns Ben; the same query with the plain string `'6a93ea08a1b2c3d4e5000002'` returns null. Switch the data view to **Extended JSON** to see how the same documents look as plain JSON.

Common mistake. Querying with the id string from the URL: `tasks.findOne({ _id: req.params.id })`. A string is never equal to an ObjectId, so this always finds nothing and the route answers 404 for every task. Convert first: `{ _id: new ObjectId(req.params.id) }`, after checking the id is well-formed (24 hexadecimal characters, e.g. with `ObjectId.isValid(id)`), otherwise `new ObjectId()` throws.

Flexible schema, and who checks the data

MongoDB does not require the documents of a collection to have the same fields or types; that flexibility moves the job of **validating** data out of the table definition and into your code, a **schema validator** on the collection, or a library.

Where the rules can live

Place	Who refuses a bad document
Your API	Your code, before writing: 400 Bad Request (see Validation)
A <code>\$jsonSchema</code> validator	MongoDB itself, on every write: Document failed validation (this card's code example)
An ODM library	Your Node process, before sending: for example Mongoose

mongosh

```
// A validator on the collection: MongoDB itself rejects bad documents.
db.createCollection('tasks', {
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['title', 'done'],
      properties: {
        title: { bsonType: 'string', minLength: 1 },
        done: { bsonType: 'bool' },
        priority: { bsonType: 'number', minimum: 1, maximum: 5 },
        tags: { bsonType: 'array', items: { bsonType: 'string' } },
      },
    },
  },
});

db.tasks.insertOne({ title: 'Study', done: 'no' });
// MongoServerError: Document failed validation
```

Example. Without any validation, `db.tasks.insertOne({ title: 'Buy bread', done: 'false' })` is accepted. The task never appears in `find({ done: false })` (its done is the **string** 'false') nor in a search on title. With this card's validator, the same insert fails at once; with validation in the API, the request is answered 400 before the database is touched.

Common mistake. Taking "schema-less" as "no design needed". The database will not stop you from writing inconsistent documents, so you find them in production. Write the shape of each collection down (fields, types, required ones) before coding, and validate in the API.

CRUD operations

Creating documents: insertOne and insertMany

`insertOne(doc)` adds one document and `insertMany([docs])` several; if a document has no `_id`, one is generated,

and the result reports the new id(s).

- **insertMany stops at the first error** (a duplicate `_id`, or a duplicate email under a unique index), and the documents before it **stay inserted**. It is not all-or-nothing; that would need a transaction.
- **No checks by default:** fields are not validated unless a validator or your code does it (see [Flexible schema](#)).
- **Dates as dates:** `ISODate(...)` in the shell, `Date()` in Node.js. Dates stored as strings sort as text.

mongosh

```
db.tasks.insertOne({
  title: 'Prepare the slides',
  done: false,
  priority: 2,
  tags: ['teaching'],
  owner: { name: 'Cleo', email: 'cleo@example.com' },
  createdAt: ISODate('2026-09-11')
})
// { acknowledged: true, insertedId: ObjectId('...') }

db.users.insertMany([
  { name: 'Dani', email: 'dani@example.com', role: 'student' },
  { name: 'Eva', email: 'eva@example.com', role: 'student' }
])
// { acknowledged: true, insertedIds: { '0': ObjectId('...'), '1': ObjectId('...') } }
```

Example. In the playground, load the example "Insert a task" and run it: the count of tasks in the data view goes from 10 to 11 and the new document starts with a generated `_id`. "The same in SQL" explains why there is no single INSERT: the tags array would be rows of another table.

Common mistake. Passing an array to `insertOne` (use `insertMany`), or putting update operators in an insert: `insertOne({ $set: { done: true } })` is rejected because field names cannot start with `$`. `$set` belongs in `updateOne`.

Reading: find, findOne and the filter

`find(filter, projection)` **returns every document that matches the filter (an object of conditions)**, `findOne` **returns the first match or null**, and `countDocuments(filter)` **returns how many match**.

find(filter) Every matching document, as a cursor.

findOne(filter) The first match, or null.

countDocuments(filter) How many documents match.

- **Value and type:** `{ priority: 5 }` does not match `priority: '5'` (a string), and `{ done: 'false' }` does not match `done: false`.
- **Missing matches** `null`: a missing field never equals a value, but `{ due: null }` finds tasks with `due: null` **and** tasks with no `due` at all.
- **A cursor, not an array:** `find` returns a pointer that fetches documents in batches as you read them. The shell prints the first 20; in Node.js you call `.toArray()` (or loop with `for await`), and chain `.sort()`, `.skip()` and `.limit()` before reading.

mongosh

```
// every open task
db.tasks.find({ done: false })
// AND: both conditions
db.tasks.find({ done: false, 'owner.name': 'Ana' })
// one document, or null
db.tasks.findOne({ title: 'Buy milk' })
db.tasks.countDocuments({ done: true })
// → 3
```

```
// Node.js driver (inside an async function)
const open = await db.collection('tasks').find({ done: false }).toArray();
```

Example. With the seed data, `db.tasks.find({ done: false })` returns 7 documents and `db.tasks.countDocuments({ done: false })` returns 7. The playground translates the filter to SQL: `SELECT * FROM tasks WHERE done = FALSE`; Now try `db.tasks.find({ done: 'false' })`: 0 documents, and a note explains why.

Common mistake. Writing the filter as SQL or JavaScript: `find({ done = false })`, `find({ priority > 3 })` or `find(done: false)`. A filter is always an **object** of field: condition pairs; comparisons other than equality use operators such as `{ priority: { $gt: 3 } }` (see [Query operators](#)).

Updating: updateOne, updateMany and update operators

`updateOne(filter, update)` **changes the first matching document and** `updateMany` **every match; the update document uses operators such as \$set, \$unset, \$inc, \$push and \$pull to say what to change.**

The update operators you will use

Operator	What it does	Example
\$set	Sets fields (creates them if missing); dot paths reach sub-documents	<code>{ \$set: { done: true, 'owner.name': 'Benjamin' } }</code>
\$unset	Removes fields (the value given is ignored)	<code>{ \$unset: { due: '' } }</code>
\$inc	Adds a number (negative to subtract); creates the field if missing	<code>{ \$inc: { priority: 1 } }</code>
\$push	Appends a value to an array (\$each for several)	<code>{ \$push: { tags: 'urgent' } }</code>
\$addToSet	Appends only if the value is not already there	<code>{ \$addToSet: { tags: 'urgent' } }</code>
\$pull	Removes every array element equal to a value or matching a condition	<code>{ \$pull: { comments: { author: 'Ben' } } }</code>

matchedCount How many documents the filter found. 0 usually means a wrong filter, and an API answers 404.

modifiedCount How many actually changed; lower when a document already had the new value.

{ upsert: true } A third argument: when nothing matches, a new document is inserted, built from the filter and the update. Use it on purpose ("create the settings document if missing").

mongosh

```
db.tasks.updateOne({ title: 'Set up CI' }, { $set: { done: true } })
// { acknowledged: true, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }

db.tasks.updateMany({ tags: 'urgent' }, { $inc: { priority: 1 } })
// matchedCount: 4, modifiedCount: 4

db.tasks.updateOne({ title: 'Set up CI' }, { done: true })
// MongoInvalidArgumentError: Update document requires atomic operators
```

Example. Marking a task done from `PATCH /tasks/:id` with body `{ "done": true }`: `await tasks.updateOne({ _id: new ObjectId(id) }, { $set: { done: req.body.done } })`, then answer 404 if `matchedCount` is 0. Running the same `$set` twice gives `matchedCount: 1, modifiedCount: 0` the second time: the task was found but already done.

Common mistake. Replacing an array when you meant to add to it: `{ $set: { tags: ['urgent'] } }` throws away the existing tags, while `{ $push: { tags: 'urgent' } }` keeps them. And `updateOne` with a filter that matches several documents changes **only the first one**: use `updateMany` when you mean all of them.

Deleting, and the danger of {}

`deleteOne(filter)` **removes the first matching document and** `deleteMany(filter)` **every match; because an empty**

filter {} matches everything, deleteMany({}) empties the collection without asking.

Habits that prevent it

- **Refuse an empty filter** before any deleteMany or updateMany.
- **Delete by _id**: one resource per request, DELETE /tasks/:id runs deleteOne({ _id }).
- **Look first**: run the filter with find or countDocuments before deleting.
- **Check the result**: a deletedCount of 0 means nothing matched, and an API answers 404.

js

```
// A clean-up helper: the guard stops the empty filter
async function deleteTasks(tasks, query) {
  const filter = {};
  if (query.done !== undefined) filter.done = query.done === 'true';
  // called without ?done= → filter is {} → EVERY task would be deleted
  if (Object.keys(filter).length === 0) {
    throw new Error('A filter is required');
  }
  const { deletedCount } = await tasks.deleteMany(filter);
  return deletedCount;
}
```

Example. In the playground, db.tasks.find({ done: true }) shows 3 tasks; db.tasks.deleteMany({ done: true }) then reports deletedCount: 3. Loading the example "Delete (empty filter!)" deletes all 10 tasks and the result warns you; press **Reset data** to bring them back.

Common mistake. Using deleteOne to mean "delete this one" with a filter that is not unique, such as { title: 'Study' }. It removes whichever matching document comes first, which may not be the one the user clicked. Identify the document by _id.

Queries

Query operators: comparison and logic

Conditions other than equality use query operators, written as { field: { \$operator: value } }: **comparison** (\$eq \$ne \$gt \$gte \$lt \$lte), **lists** (\$in \$nin), **presence** (\$exists), **patterns** (\$regex), and **\$and / \$or to combine whole conditions.**

Query operators (SQL equivalent in the last column)

Operator	Meaning	Example	SQL
\$eq / \$ne	Equal / not equal (\$ne also matches a missing field)	{ done: { \$ne: true } }	done <> TRUE
\$gt \$gte \$lt \$lte	Greater / at least / less / at most	{ priority: { \$gte: 3 } }	priority >= 3
\$in / \$nin	Equal to one of a list / to none of it	{ priority: { \$in: [1, 5] } }	priority IN (1, 5)

Operator	Meaning	Example	SQL
\$exists	The field is present (true) or absent (false)	{ due: { \$exists: true } }	due IS NOT NULL (roughly)
\$regex or /.../	A string matches a pattern (i = ignore case)	{ title: /^write/i }	title ILIKE 'write%
\$or	At least one of the filters holds	{ \$or: [{ done: true }, { priority: 5 }] }	done OR priority =
\$and	All the filters hold	{ \$and: [{ due: { \$exists: true } }, { due: { \$lt: ISODate('2026-09-26') } }] }	... AND ...

- **Same type only:** { priority: { \$gt: 3 } } never matches a priority stored as the string '5', one more reason to keep types consistent (see [Flexible schema](#)).
- **Inside or outside:** comparison operators go **inside** a field; \$and, \$or and \$nor go at the **top** of the filter and take an array of filters.
- \$and **is rarely needed:** conditions on different fields are already combined with AND. Use it only to put two conditions on the **same** field in separate objects.

mongosh

```
db.tasks.find({ priority: { $gt: 3 } })
// → 4 or 5
// a range
db.tasks.find({ priority: { $gte: 2, $lte: 4 } })
// any of these tags
db.tasks.find({ tags: { $in: ['db', 'api'] } })
// tasks that have a due date
db.tasks.find({ due: { $exists: true } })
db.tasks.find({ title: /write/i })
// → 'Write ...' in any case
db.tasks.find({ $or: [{ priority: 5 }, { done: true } ] })
```

Example. In the playground, `db.tasks.find({ $or: [{ priority: 5 }, { done: true } ] })` returns 4 tasks: Set up CI (priority 5) plus the 3 finished ones. The SQL panel shows `WHERE priority = 5 OR done = TRUE`. Mistype an operator (`$get`) and the error says `unknown operator: $get (did you mean $gt?)`.

Common mistake. Putting the operator in the wrong place: `{ $gt: { priority: 3 } }` (operator outside, field inside) fails with "unknown top level operator", and `{ priority: { $or: [...] } }` fails too. Also avoid writing the same field twice in one object (`{ priority: { $gt: 1 }, priority: { $lt: 5 } }`): in JavaScript the second silently replaces the first; combine them as `{ priority: { $gt: 1, $lt: 5 } }`.

Dot notation: inside sub-documents and arrays

Dot notation (`'owner.name'`, `'comments.author'`) reaches fields inside sub-documents and inside the elements of arrays; on an array field, a plain value matches arrays that **contain** it.

Asking about an array of values

Filter	Matches
{ tags: 'urgent' }	Arrays that contain 'urgent'
{ tags: ['devops', 'urgent'] }	Exactly that array, in that order
{ tags: { \$all: ['api', 'urgent'] } }	Contains all of them, in any order
{ tags: { \$size: 0 } }	Exactly 0 elements

Filter	Matches
<code>{ 'tags.0': 'api' }</code>	The first element is 'api'

mongosh

```
// inside a sub-document
db.tasks.find({ 'owner.name': 'Ben' })
// inside an array of sub-documents
db.tasks.find({ 'comments.author': 'Ben' })
// the array contains 'urgent'
db.tasks.find({ tags: 'urgent' })
// contains both
db.tasks.find({ tags: { $all: ['api', 'urgent'] } })
// no tags at all
db.tasks.find({ tags: { $size: 0 } })
db.tasks.find({ comments: { $elemMatch: { author: 'Ben', text: /sort/i } } })
```

Example. Against the seed data, `{ 'owner.name': 'Ben' }` finds 3 tasks, `{ 'comments.author': 'Ben' }` finds 2 (Write the SQL schema and Add pagination), and `{ tags: 'urgent' }` finds 4. The SQL panel shows why these are hard in SQL: each needs a JOIN with users, comments or task_tags.

Common mistake. Writing `{ owner: { name: 'Ben' } }` to find Ben's tasks. That asks for an owner sub-document **exactly equal** to `{ name: 'Ben' }`, with no other fields, so it finds nothing (the owner also has an email). Use dot notation for one field: `{ 'owner.name': 'Ben' }`.

Projection: choose the fields you get back

The second argument of `find` is the **projection**: `{ title: 1 }` returns only the listed fields (plus `_id`), `{ comments: 0 }` returns everything except the listed fields; the two styles cannot be mixed, except for `_id: 0`.

Inclusion: `{ title: 1 }` Only these fields, plus `_id`.

Exclusion: `{ comments: 0 }` Every field except these.

`_id: 0` Hides `_id`, in either style.

- **One style per projection:** `{ title: 1, done: 0 }` is an error ("Cannot do exclusion on field done in inclusion projection"), because nobody could tell what should happen to the other fields. `_id` is the only exception.
- **Dot paths work:** `{ 'owner.name': 1, _id: 0 }` returns `{ owner: { name: 'Ben' } }`.
- **In Node.js** the projection goes in the options: `find(filter, { projection: { title: 1 } })`, or `find(filter).project({ title: 1 })`.

mongosh

```
// _id and title
db.tasks.find({}, { title: 1 })
// only title
db.tasks.find({}, { title: 1, _id: 0 })
// everything except comments and owner
db.tasks.find({}, { comments: 0, owner: 0 })
db.tasks.find({}, { title: 1, done: 0 })
// MongoServerError: Cannot do exclusion on field done in inclusion projection

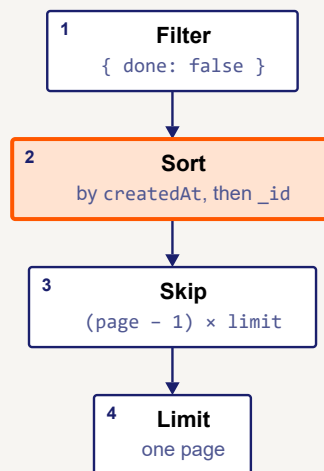
// Node.js driver
await tasks.find({ done: false }, { projection: { title: 1, _id: 0 } }).toArray();
```

Example. `db.tasks.find({ done: false }, { title: 1, _id: 0 })` returns 7 small documents such as `{ title: 'Buy milk' }`, the MongoDB version of `SELECT title FROM tasks WHERE done = FALSE`. An exclusion projection has no SQL twin: SQL cannot say "all columns except", which the playground explains under the result.

Common mistake. Passing the projection as the **first** argument: `find({ title: 1 })` is a **filter** ("documents whose title equals 1") and returns nothing. To project every document, give an empty filter first: `find({}, { title: 1 })`.

Sorting and pagination: sort, skip, limit

`.sort({ field: 1 })` orders results (1 ascending, -1 descending, several keys allowed), `.skip(n)` jumps over the first n and `.limit(n)` keeps at most n; together they give pages: `skip((page - 1) * limit).limit(limit)`.



MongoDB always sorts, then skips, then limits, whatever order you chain them in.

- **Always sort:** without sort, documents come in **natural order** (roughly the order they were stored), which MongoDB does not promise to keep, so pages could overlap or skip items.
- **Add a tie-breaker:** when the sort key can repeat, add a unique field such as `_id`: `.sort({ priority: -1, _id: 1 })`.
- **The total** comes from `countDocuments(filter)` with the same filter. The API sends the page as an array and the total in the X-Total-Count header.
- **Deep pages are slow:** skip still walks over every skipped document. Large apps page by range instead ("tasks created before the last one I showed").

js

```
// GET /tasks?page=2&limit=4 (oldest first: an array body, the total in a header)
const page = Number(req.query.page) || 1;
// cap the page size
const limit = Math.min(Number(req.query.limit) || 10, 50);
const filter = { done: false };

const [items, total] = await Promise.all([
  tasks.find(filter).sort({ createdAt: 1, _id: 1 })
    .skip((page - 1) * limit).limit(limit).toArray(),
  tasks.countDocuments(filter),
]);
res.set('X-Total-Count', String(total)).json(items);
```

Example. The seed has 10 tasks. With 4 per page sorted by `createdAt`, page 1 is tasks 1–4, page 2 is `db.tasks.find().sort({ createdAt: 1 }).skip(4).limit(4)` (Add pagination, Review pull request, Draft project plan, Design the Mongo schema), and page 3 holds the last 2. `Math.ceil(10 / 4)` gives 3 pages.

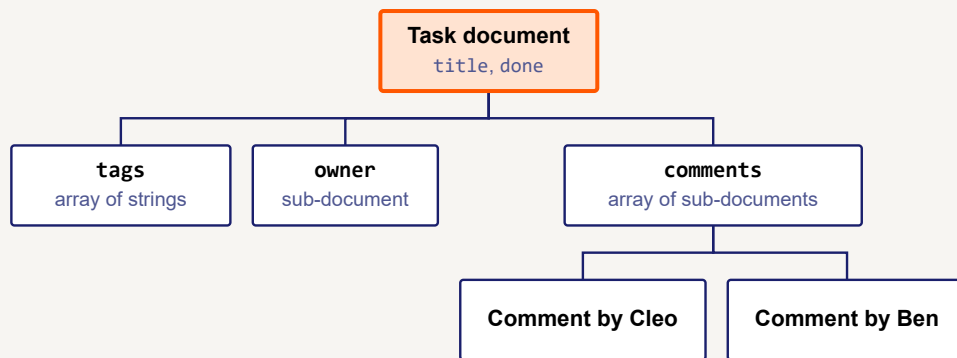
Common mistake. Computing `skip(page * limit)`, which makes page 1 start at the second page. Page n skips the $(n - 1) \times \text{limit}$ documents of the pages before it. Also check the input: `Number('abc')` is NaN, and a client asking for `limit=100000` should be capped.

Embedding and referencing

Embedding: keep together what you read together

Embedding stores related data inside the parent document, as a **sub-document** or an **array**, so one read

returns everything and one write updates it atomically.



One document holds the task with its tags, owner and comments: one read returns it all.

One read

findOne returns the task with its tags and comments: no join.

Atomic writes

A write to one document is all-or-nothing: changing done and pushing a comment in the same updateOne can never half-happen. Across several documents you would need a **transaction**.

Reaching inside

Queries use dot notation: 'comments.author' (see [Dot notation](#)).

mongosh

```

// tasks: comments and tags are embedded (they belong to this task only)
{
  _id: ObjectId('6a9bda10a1b2c3d4e5000105'),
  title: 'Add pagination',
  done: false,
  tags: [ 'api', 'urgent' ],
  comments: [
    { author: 'Cleo', text: 'Use skip and limit' },
    { author: 'Ben', text: 'Sort first!' }
  ]
}

// one atomic write changes the task and its comments together
db.tasks.updateOne(
  { title: 'Add pagination' },
  { $set: { done: true }, $push: { comments: { author: 'Ana', text: 'Done!' } } }
)

```

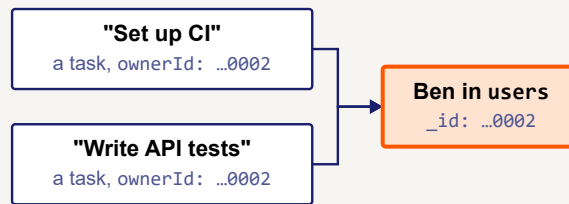
Example. In SQL, showing "Add pagination" with its tags and comments takes three tables and two joins (tasks, task_tags, comments). In MongoDB it is `db.tasks.findOne({ title: 'Add pagination' })`: one document, one round trip. Try this card's update in the playground and look at the task in the data view.

Common mistake. Embedding something that is also needed on its own. If the API has `GET /comments?author=Ben` across all tasks, or comments can be edited from a moderation page, embedded comments make those queries awkward (you must search inside every task). Embed only what is read through its parent.

Referencing: store the _id of another document

Referencing stores only the `_id` of a related document (for example `ownerId` in a task), keeping each entity in

its own collection; you fetch the related document with a second query or with \$lookup.



Many tasks point to one user, who is stored once, in their own collection.

- **Nothing checks it:** there is no REFERENCES users(id) and no ON DELETE CASCADE (see [Foreign keys](#)). A task can point at a user that does not exist, and deleting a user leaves its tasks behind: your code deletes or reassigns them (deleteMany({ ownerId: id })), ideally where it deletes the user.
- **Two reads to show both:** find the task, then findOne its user, or join them with \$lookup (see [\\$unwind and \\$lookup](#)). Both cost more than reading one embedded document, so references are for data that is shared or grows large.

js

```
// users
{ _id: ObjectId('6a93ea08a1b2c3d4e5000002'), name: 'Ben', email: 'ben@example.com' }

// tasks reference their owner by _id (like tasks.user_id in SQL)
{ _id: ObjectId('...'), title: 'Set up CI', ownerId: ObjectId('6a93ea08a1b2c3d4e5000002') }

// Node.js: two queries
const task = await db.collection('tasks').findOne({ _id: taskId });
const owner = await db.collection('users').findOne({ _id: task.ownerId });
```

Example. A user may own hundreds of tasks over a year, and the API reads tasks one by one (GET /tasks/:id) and page by page (GET /tasks?page=3&limit=10). Keeping tasks in their own collection with an ownerId reference serves both, and db.tasks.find({ ownerId: benId }).sort({ createdAt: -1 }).limit(10) (with an index on ownerId) lists Ben's latest tasks.

Common mistake. Assuming MongoDB cleans up like ON DELETE CASCADE. After db.users.deleteOne({ _id: benId }), Ben's tasks are still there, pointing at nobody, and the API may crash when it tries to show their owner. Deleting a parent means deleting or updating its children in your own code.

Embed or reference? Rules of thumb

Embed data that is read with its parent, belongs to it alone and stays small; reference data that grows without limit, is queried on its own, or is shared by many documents. A document can never exceed 16 MB.

- Is it read with the parent?** If every screen that shows a task also shows its tags: embed.
- How many can there be?** A few, with a natural limit (tags, an address, the lines of an order): embed. Hundreds, or "it keeps growing" (a user's tasks, log entries): reference.
- Is it used alone or shared?** If it has its own URL, is edited on its own, or appears on many parents (a user on many tasks): reference.

Decisions for a tasks app

Related data	Typical size	Read with parent?	Used alone?	Choice
A task's tags	A few strings	Always	No	Embed an array
A task's comments	A handful	Yes, on the task page	Rarely	Embed an array of sub-documents
A user's tasks	Grows forever	Only in pages	Yes: GET /tasks/:id	Reference: a tasks collection with ownerId

Related data	Typical size	Read with parent?	Used alone?	Choice
The owner's name on a task	One field	Yes, in lists	–	Embed a copy and keep the reference (accept duplication)
Users ↔ projects (many-to-many)	Many on both sides	Partly	Yes	Reference: arrays of ids, or a membership collection

- **16 MB per document** is a hard limit. Long before it, a huge document is slow: every read loads it whole and every \$push rewrites it. An array that grows without bound will eventually hit the limit.
- **The middle way, a copy:** embed the few fields you show often and keep the reference for the rest. The playground's tasks embed owner: { name, email }, so a list shows the owner without a second query. The price is **duplication**: if Ben changes his name, every copy must be updated (updateMany({ 'owner.email': 'ben@example.com' }, { \$set: { 'owner.name': 'Benjamin' } })) or it goes stale. Copy only fields that rarely change.

Example. A tempting first design is a user document with an embedded tasks array. By the three questions, that is a poor fit for a tasks API: tasks grow without limit, have their own routes, are paginated and are updated one at a time. A separate tasks collection with ownerId (plus, if you like, an embedded owner.name copy) follows the rules.

Common mistake. Deciding by habit: "MongoDB means embed everything" or "always normalise like SQL". Both extremes hurt. Decide per relationship, from how the data is read and how big it can grow.

Aggregation

The aggregation pipeline

An **aggregation pipeline** is an array of **stages** (\$match, \$project, \$group, \$sort, \$limit...) that documents flow through in order, each stage transforming the output of the previous one.



Each stage works only on what the stage before it handed over.

The stages you will use most

Stage	What it does	SQL idea
\$match	Keeps the documents that match a filter (same syntax as find)	WHERE (or HAVING after \$group)
\$project	Keeps, removes, renames or computes fields	The SELECT list
\$group	One output document per group, with accumulators	GROUP BY + COUNT/SUM/AVG
\$sort / \$skip / \$limit	Order, skip, keep the first n	ORDER BY / OFFSET / LIMIT
\$unwind	One document per element of an array	Joining a child table
\$lookup	Adds the matching documents of another collection as an array	LEFT JOIN
\$count	Replaces everything by one document with the number of documents	SELECT COUNT(*)

- **When to use it:** find can filter, project, sort and page. Use aggregate to **compute**: totals per group, new fields, one document per array element, data from another collection. The output documents need not look like the stored ones.
- **'\$priority' is a value:** inside stages, a string that starts with \$ means the value of that field; '\$owner.name' reaches into a sub-document.
- **\$match first:** every later stage then handles fewer documents, and only a leading \$match can use an index.

mongosh

```
// The 3 open tasks with the highest priority, as small documents
db.tasks.aggregate([
  { $match: { done: false } },
```

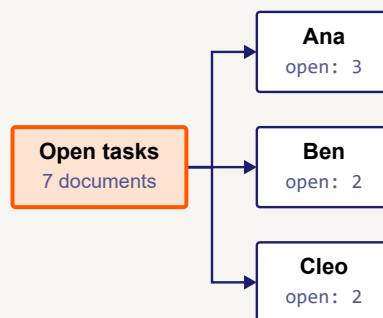
```
// → 10 → 7 documents
// order them
{ $sort: { priority: -1, createdAt: 1 } },
{ $limit: 3 },
// → 7 → 3
{ $project: { _id: 0, title: 1, owner: '$owner.name', tagCount: { $size: '$tags' } } }
})
// { title: 'Set up CI', owner: 'Ben', tagCount: 2 }
// { title: 'Write the SQL schema' ... } is not here: it is done.
```

Example. Run this card's pipeline in the playground and then remove one stage at a time: without `$limit` you get 7 documents, without `$match` the finished tasks appear, without `$project` you get whole documents. Watching what each stage receives is the fastest way to debug a pipeline.

Common mistake. Passing the stages without the array: `aggregate({ $match: ... }, { $group: ... })`. The pipeline is **one array** of stage objects, and each stage object has exactly one key: `aggregate([ { $match: ... }, { $group: ... } ])`.

\$group: one result per group

`$group` collects documents that share the same `_id` expression into one output document per group, and computes accumulators for each group: `$sum`, `$avg`, `$min`, `$max`, `$push`, `$addToSet`, `$count`.



The 7 open tasks become one result per owner.

`_id: '$owner.name'`

The group key: one group per owner name. `_id: null` makes one group of everything (totals for the whole collection).

`{ $sum: 1 }`

Adds 1 per document: a count.

`{ $sum: '$priority' }, { $avg: '$priority' }`

The total and the average of a field; `$min` and `$max` work the same way.

`{ $push: '$title' }`

Collects the values into an array (`$addToSet` skips duplicates).

- **Only `_id` and the accumulators survive:** every other field of the input documents is gone.
- **Filter groups** by their totals (SQL's `HAVING`) with a `$match` **after** the `$group`; order them with a `$sort` on the new field.

mongosh

```
// Open tasks per owner, with their average priority, busiest first
db.tasks.aggregate([
  { $match: { done: false } },
  { $group: {
    _id: '$owner.name',
    open: { $sum: 1 },
    avgPriority: { $avg: '$priority' },
    titles: { $push: '$title' }
  } },
  { $sort: { open: -1 } }
])
```

```
// { _id: 'Ana', open: 3, avgPriority: 2, titles: [ 'Buy milk', ... ] }
// { _id: 'Ben', open: 2, avgPriority: 4, titles: [ 'Set up CI', 'Write API tests' ] }
// { _id: 'Cleo', open: 2, avgPriority: 3.5, titles: [ ... ] }
```

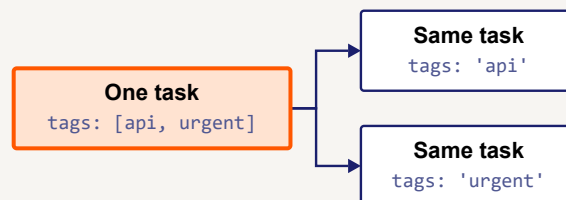
```
-- The same in SQL (relational tasks/users schema)
SELECT u.name, COUNT(*) AS open, AVG(t.priority) AS avg_priority
FROM tasks t JOIN users u ON u.id = t.user_id
WHERE t.done = FALSE
GROUP BY u.name
ORDER BY open DESC;
```

Example. An "open tasks per user" pipeline over a different data shape is the same idea (tasks embedded in users): \$unwind the tasks, \$match the open ones, then \$group: { _id: '\$email', openTasks: { \$sum: 1 } }. In the playground, challenge "Tasks per owner" asks for { _id: <name>, count: <number> } per owner.

Common mistake. Forgetting the \$ in the group key: _id: 'owner.name' is the constant text "owner.name", so every document lands in **one** group and you get a single result with the total count. Field paths in aggregation expressions always start with \$. (In a \$match filter, by contrast, field names never take a \$.)

\$unwind and \$lookup

\$unwind: '\$tags' **turns one document with an array into one document per element**; \$lookup **adds, to each document, an array of the matching documents of another collection (a left outer join).**



\$unwind copies the document once for each element of its array.

{ \$unwind: '\$tags' } One document per array element: a task tagged ['api', 'urgent'] becomes two documents, identical except that tags is 'api' in one and 'urgent' in the other. A \$group by '\$tags' can then count tasks per tag.

\$lookup Adds to each document an **array** of the matching documents of another collection. You name the collection (from), the field in this document (localField), the matching field there (foreignField) and the new field to fill (as).

- **Empty arrays vanish:** documents whose array is empty or missing disappear after \$unwind, unless you write { \$unwind: { path: '\$tags', preserveNullAndEmptyArrays: true } }.
- **A left outer join:** every input document stays after \$lookup, with an empty as array when nothing matches, like SQL's LEFT JOIN, except that the matches arrive as an array inside the document instead of as extra rows.
- **A \$lookup on every request** hints that the data might be better embedded (see [Embed or reference?](#)). When you do use it, index the foreignField, or every lookup scans the other collection.

mongosh

```
// Tasks per tag
db.tasks.aggregate([
  { $unwind: '$tags' },
  // → 10 tasks → 17 (task, tag) documents
  { $group: { _id: '$tags', n: { $sum: 1 } } },
  { $sort: { n: -1, _id: 1 } }
])
// { _id: 'urgent', n: 4 }, { _id: 'db', n: 3 }, { _id: 'api', n: 2 }, ...

// Each task with its owner's full user document
```

```
db.tasks.aggregate([
  { $match: { title: 'Set up CI' } },
  { $lookup: { from: 'users', localField: 'owner.email', foreignField: 'email', as: 'ownerDoc' } }
])
// { ..., title: 'Set up CI', ownerDoc: [ { name: 'Ben', role: 'student', ... } ] }
```

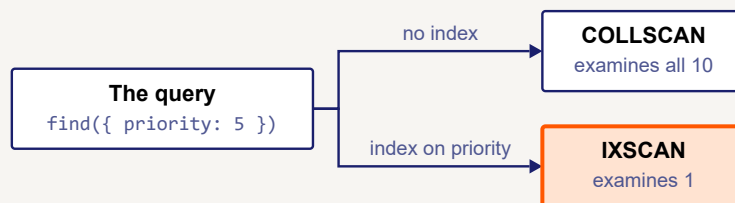
Example. In the seed, 9 tasks have tags (17 tags in total) and "Book a room for the demo" has none. `[{ $unwind: '$tags' }]` alone returns 17 documents; "Book a room" is not among them. With `preserveNullAndEmptyArrays: true` you get 18.

Common mistake. Expecting `$lookup` to return one joined object: `ownerDoc` is always an **array**, even for exactly one match, so `ownerDoc.name` in your code is undefined. Read `ownerDoc[0]`, or add `{ $unwind: '$ownerDoc' }` after the lookup.

Indexes and the back end

Indexes: from scanning to looking up

An **index** is a sorted structure (a **B-tree**) of the values of one or more fields that points to the documents; with it, MongoDB finds matches without reading every document (**IXSCAN** instead of **COLLSCAN**), at the cost of storage and slower writes.



With an index on priority, the same query reads 1 document instead of 10.

`createIndex({ priority: 1 })`

An index on one field: 1 ascending, -1 descending. Every collection already has a unique index on `_id`.

Compound: `{ ownerId: 1, createdAt: -1 }`

Serves "Ben's tasks, newest first", and queries on `ownerId` alone (a **prefix**), but not queries on `createdAt` alone.

Unique: `{ email: 1 }, { unique: true }`

The database refuses duplicates: one account per email. A duplicate fails with error **E11000**.

`.explain('executionStats')`

Shows the plan: **COLLSCAN** reads every document, **IXSCAN** uses an index. Compare `totalDocsExamined` with `nReturned`: examining 10 000 documents to return 10 is a missing index.

mongosh

```
db.tasks.find({ priority: 5 }).explain('executionStats')
// winningPlan: { stage: 'COLLSCAN' } · totalDocsExamined: 10 · nReturned: 1

db.tasks.createIndex({ priority: 1 })
db.tasks.find({ priority: 5 }).explain('executionStats')
// winningPlan: FETCH ← IXSCAN { priority: 1 } · totalDocsExamined: 1 · nReturned: 1

// compound: owner, then newest first
db.tasks.createIndex({ ownerId: 1, createdAt: -1 })
// one account per email
db.users.createIndex({ email: 1 }, { unique: true })
db.users.insertOne({ name: 'Ana 2', email: 'ana@example.com' })
// MongoServerError: E11000 duplicate key error ... index: email_1 dup key: { email: 'ana@example.com' }
```

Example. In the playground, load the example "Index + explain": the first plan is a COLLSCAN that examines all 10 tasks; after `createIndex({ priority: 1 })` the same query is an IXSCAN that examines only the 1 matching task. (The playground's planner is simplified, but the before/after is the real behaviour.) The example "Unique index" shows the E11000 error.

Common mistake. Indexing "just in case". An index on `email` makes `find({ email })` faster, but makes every **insert**, every **update** that changes `email`, and every **delete** slower, and uses memory. Add an index when a real, frequent query needs it, and check with `explain`.

Connecting from Node.js: the official driver

The official `mongodb` package gives you a `MongoClient` that you **connect once at startup and reuse**; `client.db('app').collection('tasks')` **returns a collection with the same methods as the shell, all returning Promises**.

From `mongosh` to the Node.js driver

	mongosh	Node.js driver
Results	Printed	Promises: await them inside async functions
<code>find</code>	Prints the first 20	A cursor: finish it with <code>.toArray()</code>
Dates	<code>ISODate(...)</code>	<code>new Date(...)</code>
Ids	<code>ObjectId(...)</code>	<code>new ObjectId(...)</code> , imported from <code>mongodb</code>

js

```
// db.js: connect once, share the database object
const { MongoClient } = require('mongodb');
const client = new MongoClient(process.env.MONGO_URL);
let db;
async function connect() {
  await client.connect();
  db = client.db('app');
  return db;
}
module.exports = { connect, getDb: () => db };

// server.js: no top-level await in CommonJS, so wait with .then
const app = require('./app');
const { connect } = require('./db');
connect().then(() => app.listen(3000, () => console.log('API on http://localhost:3000')));

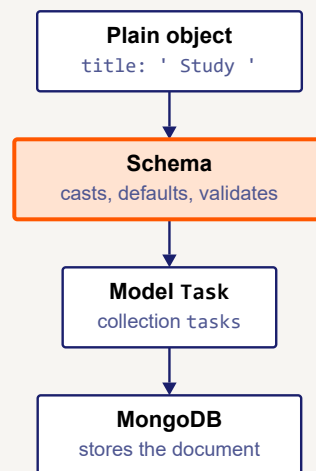
// models/tasksModel.js: the same seam as the SQL model, MongoDB inside
const { ObjectId } = require('mongodb');
function createTasksModel(db) {
  const tasks = db.collection('tasks');
  const toDto = ({ _id, ...rest }) => ({ id: _id.toString(), ...rest });
  return {
    async findById(id) {
      // bad id → treated as not found
      if (!ObjectId.isValid(id)) return null;
      const doc = await tasks.findOne({ _id: new ObjectId(id) });
      return doc ? toDto(doc) : null;
    },
    async create({ title, done = false }) {
      const doc = { title, done, createdAt: new Date() };
      const { insertedId } = await tasks.insertOne(doc);
      return toDto({ ...doc, _id: insertedId });
    },
  };
}
module.exports = { createTasksModel };
```

Example. A controller keeps calling `await tasksModel.findById(req.params.id)` and answers 404 on null, exactly as with the SQL model; only `models/` changed: swap the storage, keep the shape. The API still returns `{ "id": "6a9a...104", "title": "Set up CI", ... }`.

Common mistake. Passing request data straight into a filter: `users.findOne({ email: req.body.email, password: req.body.password }).express.json()` turns `{"password": {"$ne": null}}` into an object, the filter then means "any password that is not null", and the attacker logs in without knowing it: **NoSQL injection**. Check that values are strings (`typeof x === 'string'`, or validate the body) before building filters, and never compare plain passwords anyway: they are stored hashed.

Mongoose: schemas and models on top of the driver

Mongoose is an ODM (object-document mapper) for Node.js: you declare a **schema** (fields, types, required, defaults, limits) and get a **model** whose methods validate documents before writing them to MongoDB.



Mongoose checks and completes each document in your Node process, before MongoDB sees it.

Schema

The fields with their types and rules: required, default, min and max, trim.

Model

`mongoose.model('Task', taskSchema)`: a class bound to a collection, named in lower case and plural (tasks). `Task.find`, `Task.create` and `Task.findById` mirror the driver, cast types (an id string becomes an ObjectId) and drop fields that are not in the schema.

ref + .populate() Replaces a referenced id by its document: a second query behind the scenes, like a manual `$lookup`.

js

```

const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true, maxLength: 100 },
  done: { type: Boolean, default: false },
  priority: { type: Number, min: 1, max: 5, default: 3 },
  tags: [String],
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  // adds createdAt and updatedAt
}, { timestamps: true });

// collection 'tasks'
const Task = mongoose.model('Task', taskSchema);

// inside async functions (connect once at startup: await mongoose.connect(url))
// title -> 'Study'
const task = await Task.create({ title: ' Study ', owner: userId });
const open = await Task.find({ done: false }).sort({ createdAt: -1 }).limit(10).populate('owner', 'name');
```

```
// rejected: max 5
await Task.findByIdAndUpdate(id, { priority: 9 }, { runValidators: true });
```

Example. `await Task.create({ owner: userId })` throws a `ValidationError` ("Path title is required") without contacting the database, which your error handler can turn into `400 Bad Request`. The same insert through the bare driver would have been stored.

Common mistake. Believing Mongoose makes MongoDB itself enforce the schema. The rules live in your Node process: a script, another service or the shell that writes to the same collection bypasses them, and updates skip validation unless you ask for it. For rules that must always hold, add a `$jsonSchema` validator or a unique index in the database too.

SQL or MongoDB? Choosing honestly

Choose a **relational database** when your data is many linked entities with stable rules and you want the database to guarantee integrity; choose **MongoDB** when records are self-contained documents read as a whole, or their shape varies. Both can run a typical web API.

Which way does your project lean?

Question	Leans SQL	Leans MongoDB
What does a typical read return?	Rows combined from several tables	One document with everything inside
How stable is the shape?	Known up front, rarely changes	Varies per record or changes often
Who must enforce the rules?	The database (NOT NULL, FK, CHECK)	Your code or a validator is acceptable
Relationships	Many-to-many, shared entities, cascades	Mostly parent → few children
Reports	Ad-hoc joins and GROUP BY across entities	Pipelines over one collection
Multi-record changes	Frequent (money transfers, stock)	Rare: most writes touch one document

Claims that are out of date

- **"MongoDB has no transactions"**: multi-document **ACID transactions** exist since MongoDB 4.0 (on replica sets), although single-document writes, which are atomic, remain the normal tool.
- **"SQL cannot store JSON"**: PostgreSQL has a `jsonb` column type with indexes.
- **"MongoDB scales and SQL does not"**: both scale far beyond a small project; at that size, design and indexes matter far more than the engine.

Example. A tasks API fits both. SQL: users and tasks with a foreign key and `ON DELETE CASCADE`, owner checks by `user_id`, `LIMIT/OFFSET` pages. MongoDB: a tasks collection with `ownerId` (plus an embedded `owner.name` copy), tags and comments embedded, an index on `{ ownerId: 1, createdAt: -1 }`. A course-enrolment system with students, groups, subjects and grades, queried in every direction, leans clearly towards SQL.

Common mistake. Picking MongoDB "to avoid designing a schema". The design work does not disappear; it moves into your code, where the database can no longer help you. Pick it when the document model fits your reads, and then design the documents carefully.

From SQL to MongoDB: the vocabulary

A recap of the section as a dictionary: **table** → **collection**, **row** → **document**, **column** → **field**, **primary key** → **_id**, **JOIN** → **embedding** or **\$lookup**, **WHERE** → **a filter document**, **GROUP BY** → **\$group**.

The same ideas in both worlds (the tasks/users domain)

Idea	SQL (PostgreSQL)	MongoDB
A group of similar records	Table tasks	Collection tasks
One record	Row (flat)	Document (can nest objects and arrays)
A named value	Column (declared, one type)	Field (not declared; any type)
Identity	<code>id SERIAL PRIMARY KEY</code>	<code>_id</code> (ObjectId by default)
Related data	Foreign key <code>user_id</code> + <code>JOIN</code>	Embedded sub-document, or an <code>_id</code> reference + <code>\$lookup</code>

Idea	SQL (PostgreSQL)	MongoDB
Read	SELECT title FROM tasks WHERE done = FALSE	db.tasks.find({ done: false }, { title: 1 })
Sort and page	ORDER BY ... LIMIT 10 OFFSET 20	.sort({ ... }).skip(20).limit(10)
Count per group	SELECT user_id, COUNT(*) ... GROUP BY user_id	aggregate([{ \$group: { _id: '\$owner', n: { \$sum: 1 } } }])
Create / change / remove	INSERT, UPDATE ... SET, DELETE	insertOne, updateOne + \$set, deleteOne
Schema rules	Enforced by the database (NOT NULL, types, FK)	Enforced by your code, a validator or Mongoose

- **Where related data lives** is the deepest difference: SQL keeps each entity in its own table and joins at query time; MongoDB embeds or references (see [Embed or reference?](#)).
- **The database checks less:** no schema unless you add one (see [Flexible schema](#)), and no foreign keys: nothing stops a task from pointing at a user that was deleted.

Example. The relational query "open tasks of ana@example.com" is SELECT t.title FROM tasks t JOIN users u ON u.id = t.user_id WHERE u.email = 'ana@example.com' AND t.done = FALSE. With the owner embedded in each task, MongoDB needs no join: db.tasks.find({ 'owner.email': 'ana@example.com', done: false }, { title: 1 }). The playground shows "The same in SQL" under each result so you can compare.

Common mistake. Translating a SQL schema table by table, with a task_tags collection, a comments collection and ids everywhere, then joining them with \$lookup in every query. That keeps all the costs of the relational model and loses its guarantees (no foreign keys, weaker joins). Design documents around how the data is **read**.